Smart Coding Practice Assistant

Python, C & Java Basics

Purpose: This document provides beginner-friendly programming concepts and examples that can be used as a knowledge base for a Smart Coding Practice Assistant / RAG project.

1. Python Basics

Introduction

Python is a high-level, interpreted programming language known for simple syntax and readability. It is widely used in web development, automation, data science, artificial intelligence, and scripting.

Variables and Data Types

Variables store values. Common Python data types include int, float, str, bool, list, tuple, set, and dict.

Example

```
name = "Ravi"
age = 20
marks = 85.5
print(name)
print(age)
print(marks)
```

Conditional Statements

if, elif, and else are used to make decisions.

Example

```
age = 18
if age >= 18:
print("Eligible")
else:
print("Not eligible")
```

Loops

for and while loops repeat a block of code.

Example

```
for i in range(1, 6):
print(i)
```

Functions

A function is a reusable block of code defined using def.

Example

```
def add(a, b):
return a + b

print(add(10, 20))
```

Lists

A list is an ordered, mutable collection. Example: numbers = [10, 20, 30]. Elements can be accessed using indexes.

Dictionaries

A dictionary stores data as key-value pairs. Example: student = {'name': 'Ravi', 'age': 20}.

2. C Basics

Introduction

C is a procedural, compiled programming language widely used for system programming, embedded systems, operating systems, and foundational programming.

Basic Program Structure

A C program commonly contains header files, the main() function, declarations, statements, and a return statement.

Example

```
#include <stdio.h>

int main() {
printf("Hello World");
return 0;
}
```

Variables and Data Types

Common C data types include int, float, char, and double. Variables must generally be declared with a type.

Input and Output

printf() displays output and scanf() accepts formatted input.

Example

int age; scanf("%d", &age;); printf("Age = %d", age);

Conditional Statements

C uses if, else if, else, and switch for decision making.

Loops

for, while, and do-while loops are used for repetition.

Arrays

An array stores multiple values of the same data type in contiguous memory locations. Example: int a[5] = {10,20,30,40,50};

Functions

Functions divide a program into reusable modules. They can accept parameters and return a value.

Pointers

A pointer stores the memory address of another variable. Example: int *p = &x;

3. Java Basics

Introduction

Java is a high-level, object-oriented programming language designed around portability. Java programs are compiled into bytecode and executed by the Java Virtual Machine (JVM).

Basic Program Structure

A Java application commonly contains a class and a main() method.

Example

```
class Hello {
public static void main(String[] args) {
System.out.println("Hello World");
}
}
```

Variables and Data Types

Java has primitive types such as int, float, double, char, and boolean. It also supports reference types such as String, arrays, and objects.

Conditional Statements

Java supports if, else-if, else, and switch statements.

Loops

for, while, and do-while loops are commonly used for repetition.

Methods

A method is a block of code inside a class that performs a specific task.

Example

```
class Demo {
static int add(int a, int b) {
return a + b;
}

public static void main(String[] args) {
System.out.println(add(5, 7));
}
}
```

Object-Oriented Programming

Core OOP concepts include encapsulation, inheritance, polymorphism, and abstraction.

Arrays and Strings

Arrays store multiple values of the same type. String is a class used to represent text.

4. Quick Comparison

Python vs C vs Java

Python emphasizes readability and rapid development. C provides low-level control and high performance. Java combines object-oriented programming with platform independence through the JVM.

Compilation / Execution

Python is generally interpreted/bytecode executed through its runtime. C is compiled to native machine code. Java is compiled to bytecode and executed by the JVM.

Best Learning Focus

For beginners: learn variables, operators, conditions, loops, functions/methods, arrays/collections, debugging, and problem solving in each language.

5. Coding Practice Topics

Beginner Problems

Print a message; add two numbers; find the largest of two or three numbers; check even/odd; calculate factorial; generate a multiplication table; reverse a number; check prime number.

Intermediate Problems

Reverse a string; count vowels; find maximum and minimum in an array/list; sort numbers; search an element; remove duplicates; count character frequency; implement a simple calculator.

Problem-Solving Steps

1. Understand the problem. 2. Identify inputs and outputs. 3. Create an algorithm or pseudocode. 4. Write code. 5. Test normal and edge cases. 6. Debug errors. 7. Improve readability and efficiency.

Debugging Tips

Read the complete error message, identify the line number, check variable values, verify syntax and data types, test small parts of the program, and use meaningful variable names.

Practice Checklist

- Understand variables and basic data types.
- Write programs using if/else and switch where applicable.
- Use loops confidently.
- Create functions/methods.
- Work with arrays, lists, strings, and dictionaries where applicable.
- Understand basic OOP concepts in Java.
- Practice debugging compiler, syntax, runtime, and logic errors.
- Solve programming problems independently before checking solutions.